Alpha-beta search



Zola Mahlaza

Department of Computer Science

Feb 2026

Topics

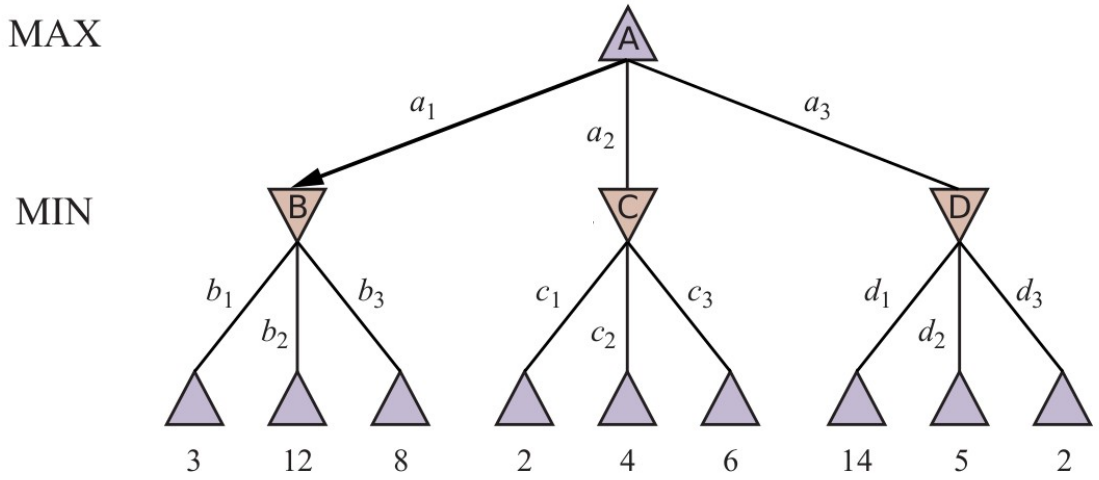
- The minimax algorithm
- Alpha-delta search

Limitation of minimax

- Time complexity is $O(b^m)$ where b = branching factor, m = max depth
  - This is significant challenge!
- We need to eliminate some of the nodes
  - No algorithm will eliminate the exponent m
  - We can prune a large number of the nodes

Alpha-delta search motivation

- Actions to be taken by each agent



- When processing this game tree, do all nodes have to be explored?

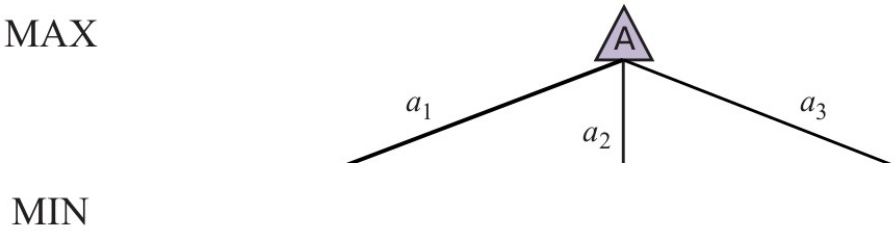
Alpha-delta search motivation

- Minimax value calculation steps:



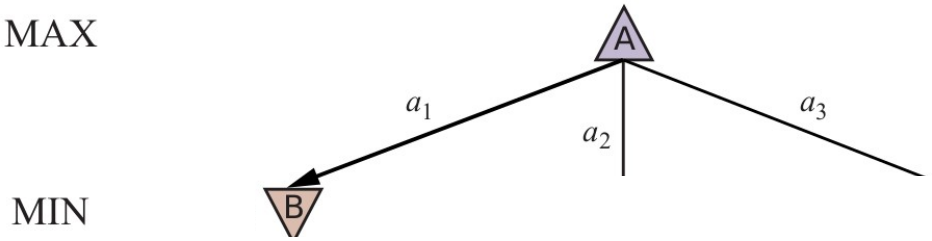
Alpha-delta search motivation

- Minimax value calculation steps:



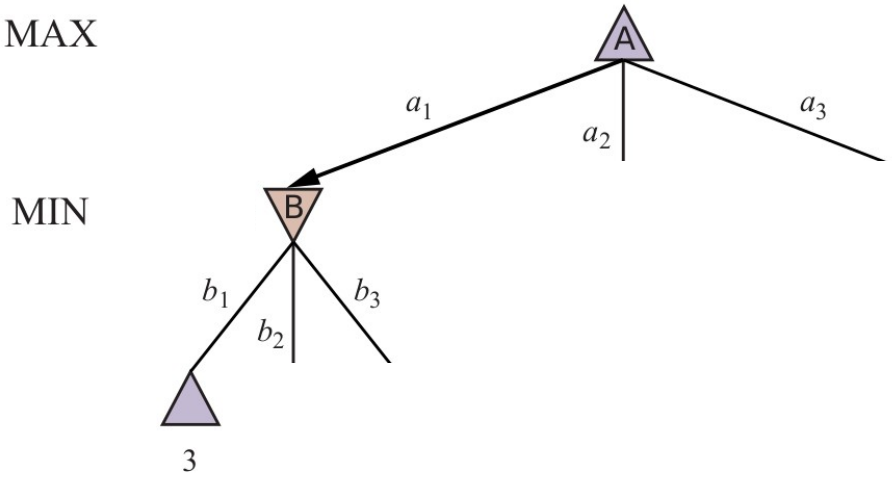
Alpha-delta search motivation

- Minimax value calculation steps:



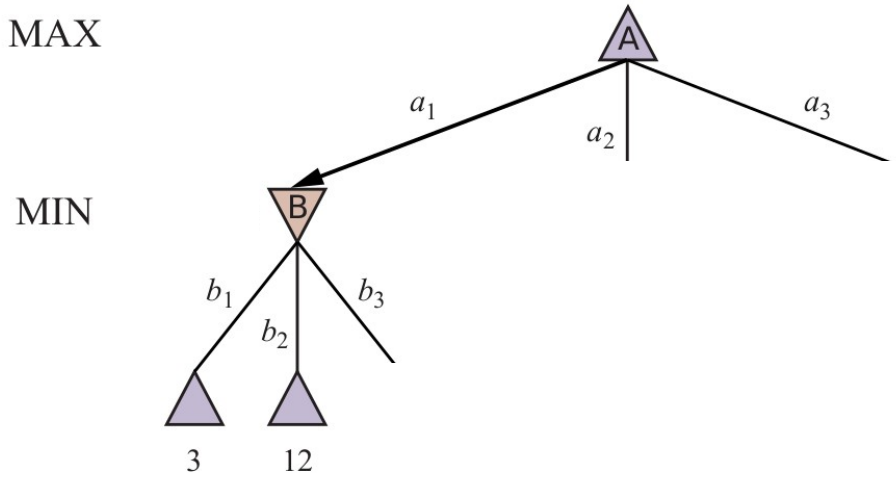
Alpha-delta search motivation

- Minimax value calculation steps:



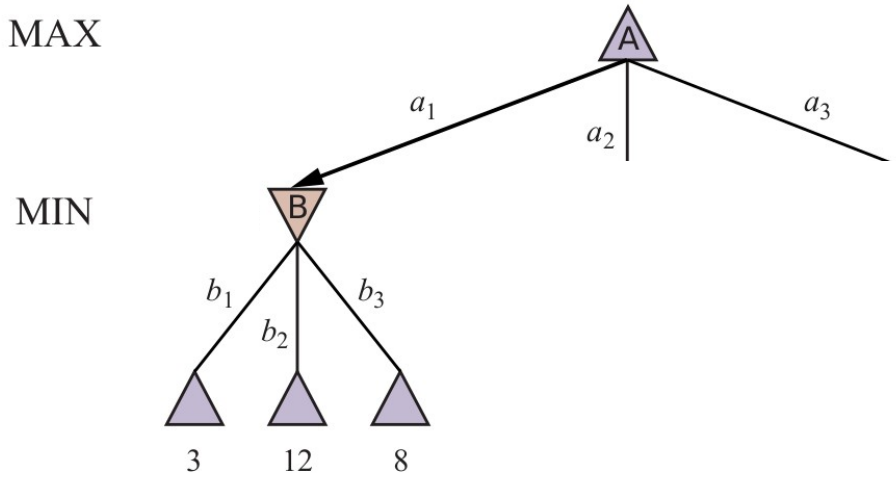
Alpha-delta search motivation

- Minimax value calculation steps:



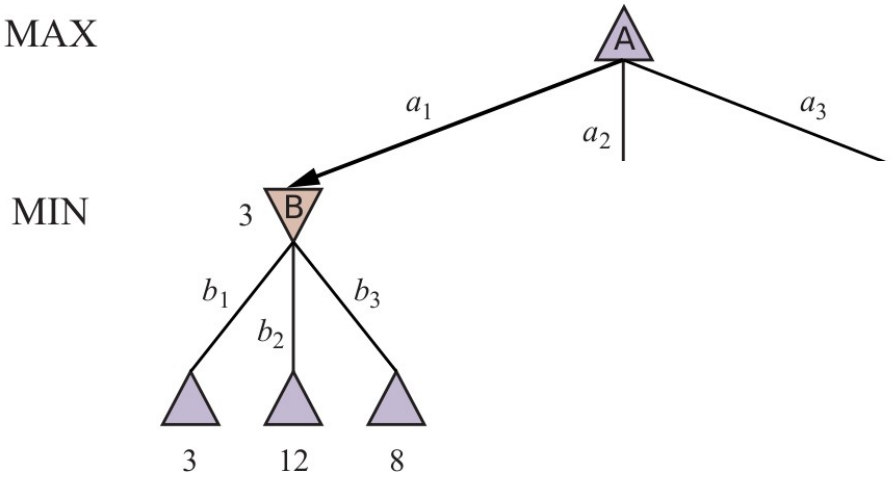
Alpha-delta search motivation

- Minimax value calculation steps:



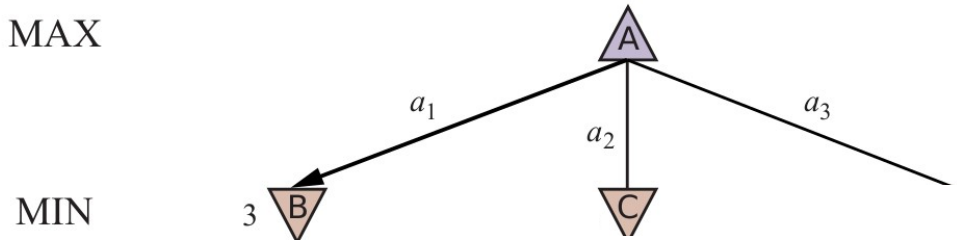
Alpha-delta search motivation

- Minimax value calculation steps:



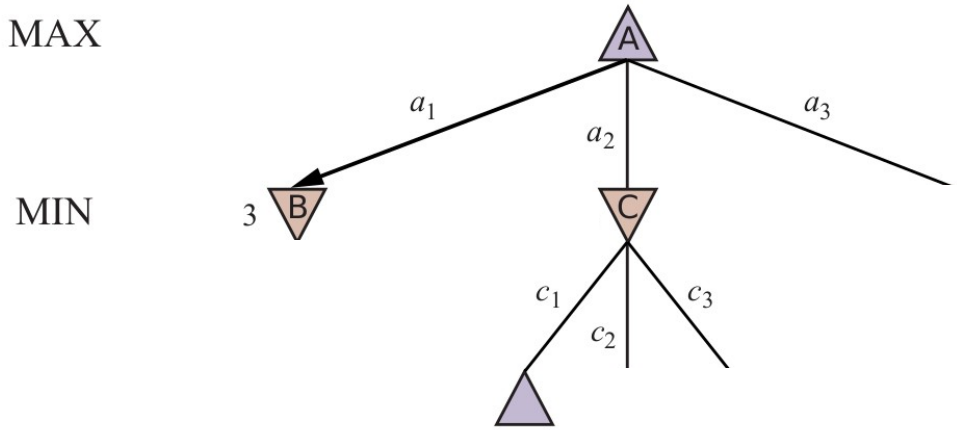
Alpha-delta search motivation

- Minimax value calculation steps:



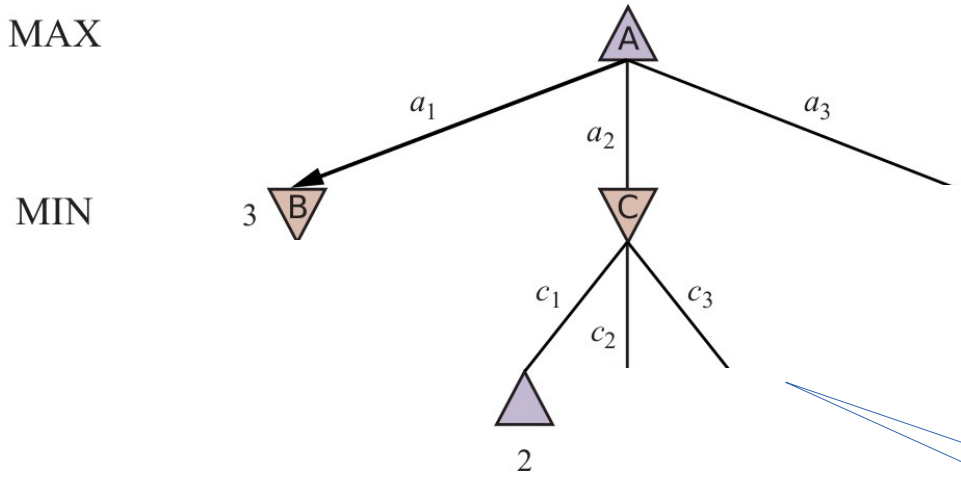
Alpha-delta search motivation

- Minimax value calculation steps:



Alpha-delta search motivation

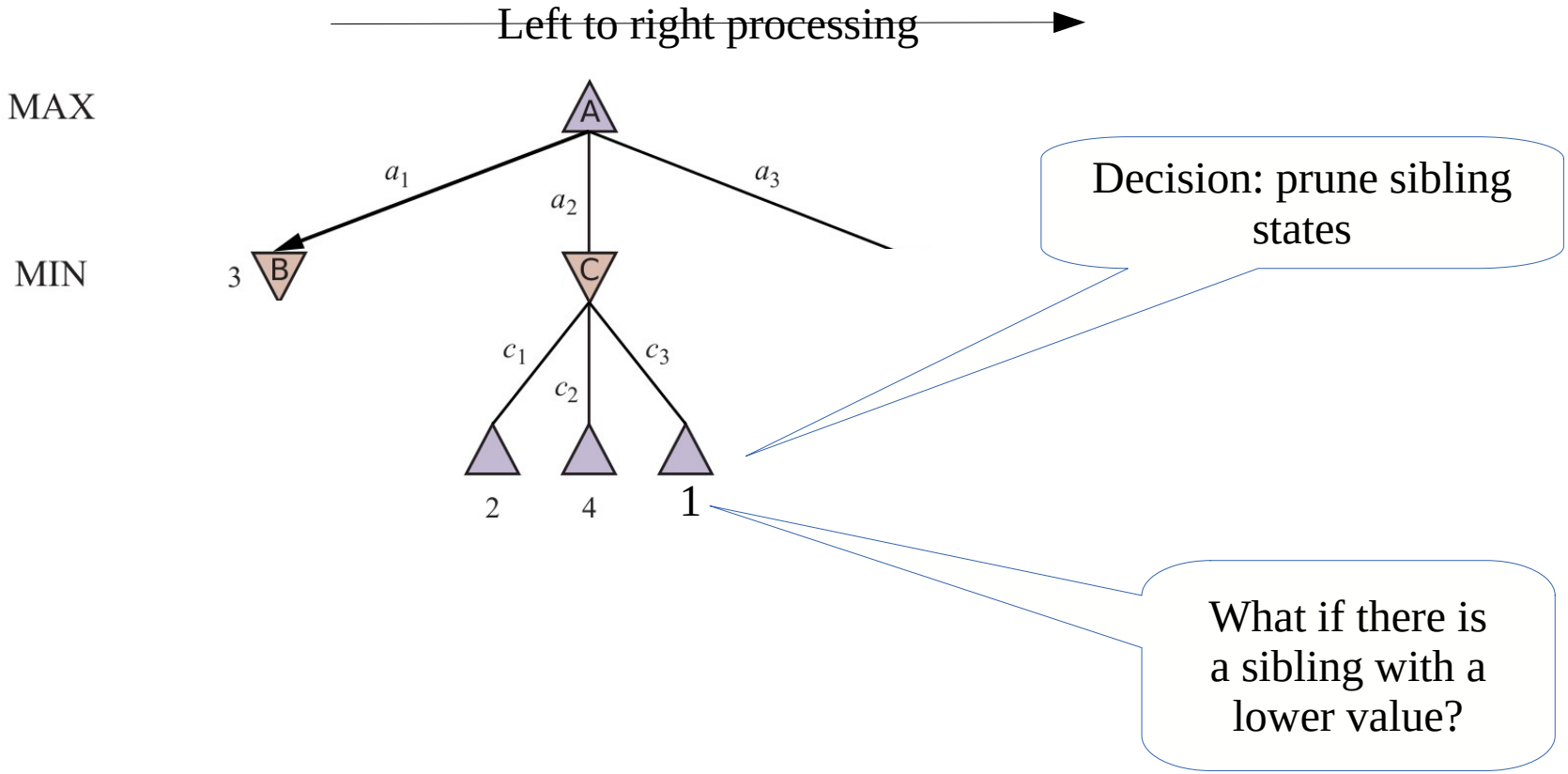
- Minimax value calculation steps:



When choosing between B and C, do the values of result(C, C_2) and result(C, C_3) matter?

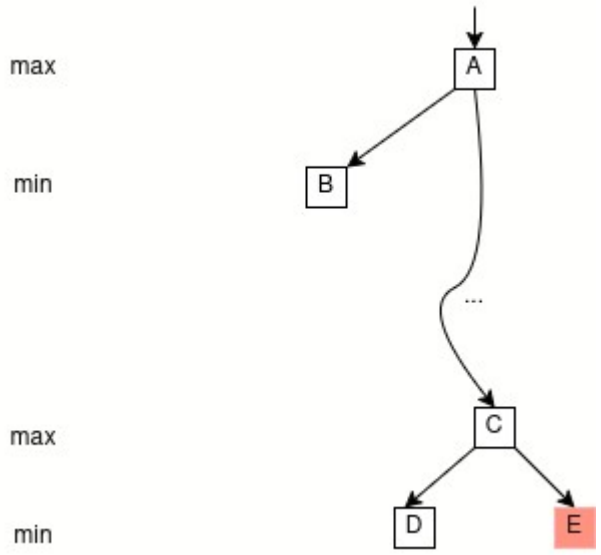
Alpha-delta search motivation

- Minimax value calculation steps:



Alpha-delta search motivation

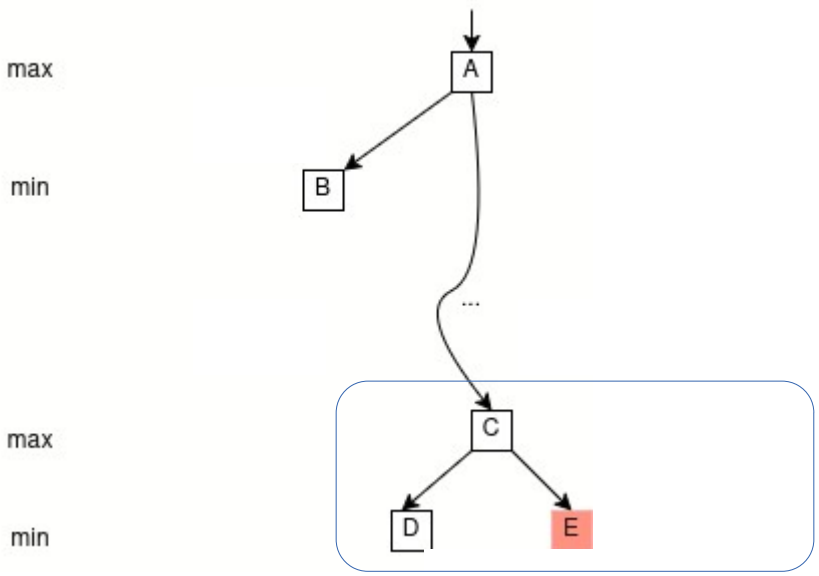
- General approach to comparison, consider the ff:



- Will E ever considered if D and B are the max. values in the respective levels?

Alpha-delta search motivation

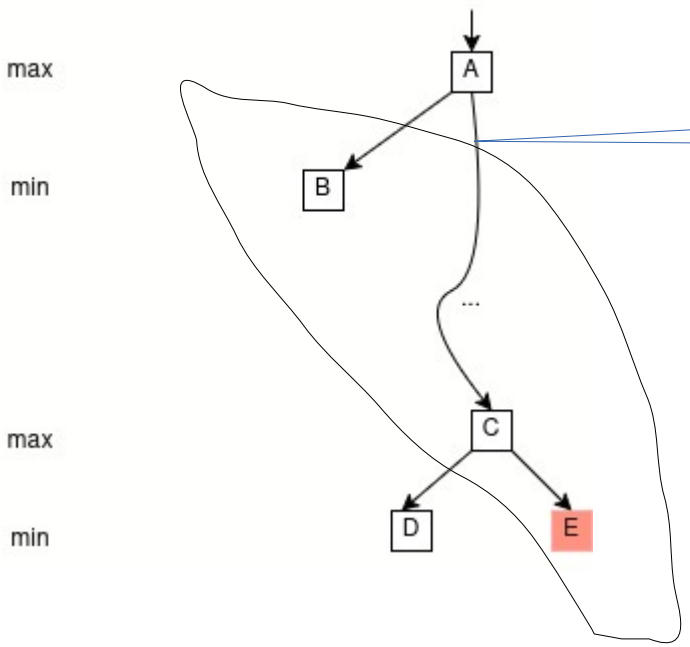
- General approach to comparison, consider the ff:



Option 1:
Same level and
D > E so choose
D always

Alpha-delta search motivation

- General approach to comparison, consider the ff:



This path/subtree would never be considered

Option 2:
B was max at top level

- Consequently, we can prune E!

Alpha-delta search motivation

- Minimax is depth-first, it considers the nodes along a single path in the tree
- We introduce two new params in (MAX/MIN)-VALUE (state, α, β) for pruning
  - This is the source of the name of the algorithm
- α = Highest-value choice found so far at any point
- β = Lowest-value choice so far at any point

Alpha-delta search I: search

We start at root of game
Tree, but can change the state

```
function ALPHA-BETA-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state, −∞, +∞)
    return move
```

Initial alpha to smallest
Values, beta to largest vaue

Alpha-delta search II: max value

Base case: terminal states rely on utility function

```
function MAX-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a), α, β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move
```

Alpha-delta search II: max value

Determine min value for opponent

```
function MAX-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a), α, β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move
```

Alpha-delta search II: max value

Keep track of the largest value

```
function MAX-VALUE(game, state, α, β) returns a (u    ove) pair
  if game.IS-TERMINAL(state) then return game.UTILI    ate, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a), α, β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move
```

Alpha-delta search II: max value

Should you encounter the value that exceeds smallest value
, stop.
Disregard other subtrees

```
function MAX-VALUE(game, state, α, β) returns , move) pair
  if game.IS-TERMINAL(state) then return game. (state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a),  β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move
```

Alpha-delta search III: min value

```
function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESUL  ate, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

Base condition: if you are at a terminal state, rely on utility function

Alpha-delta search III: minvalue

```
function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

Determine max values for player

Alpha-delta search III: min value

```
function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

Update the min value for player

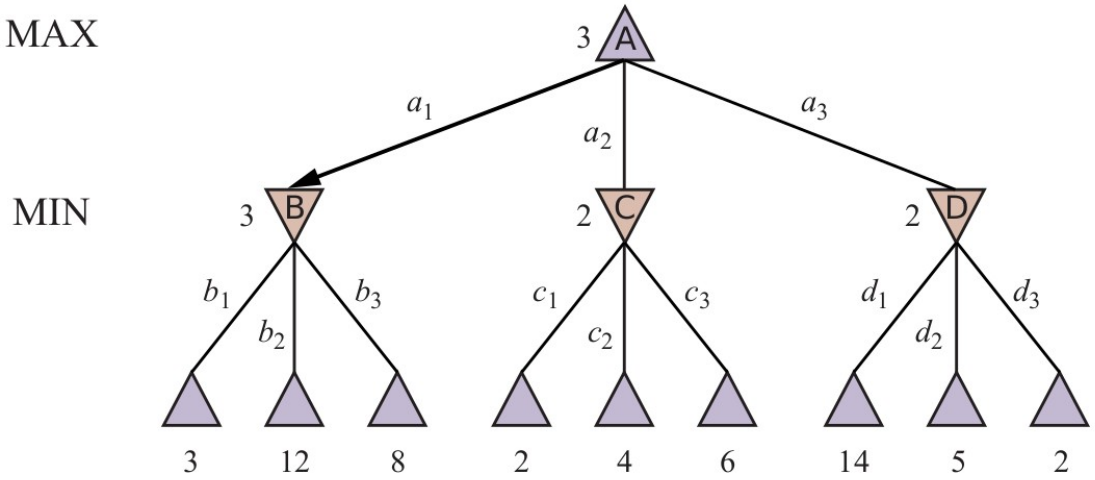
Alpha-delta search III: min value

```
function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

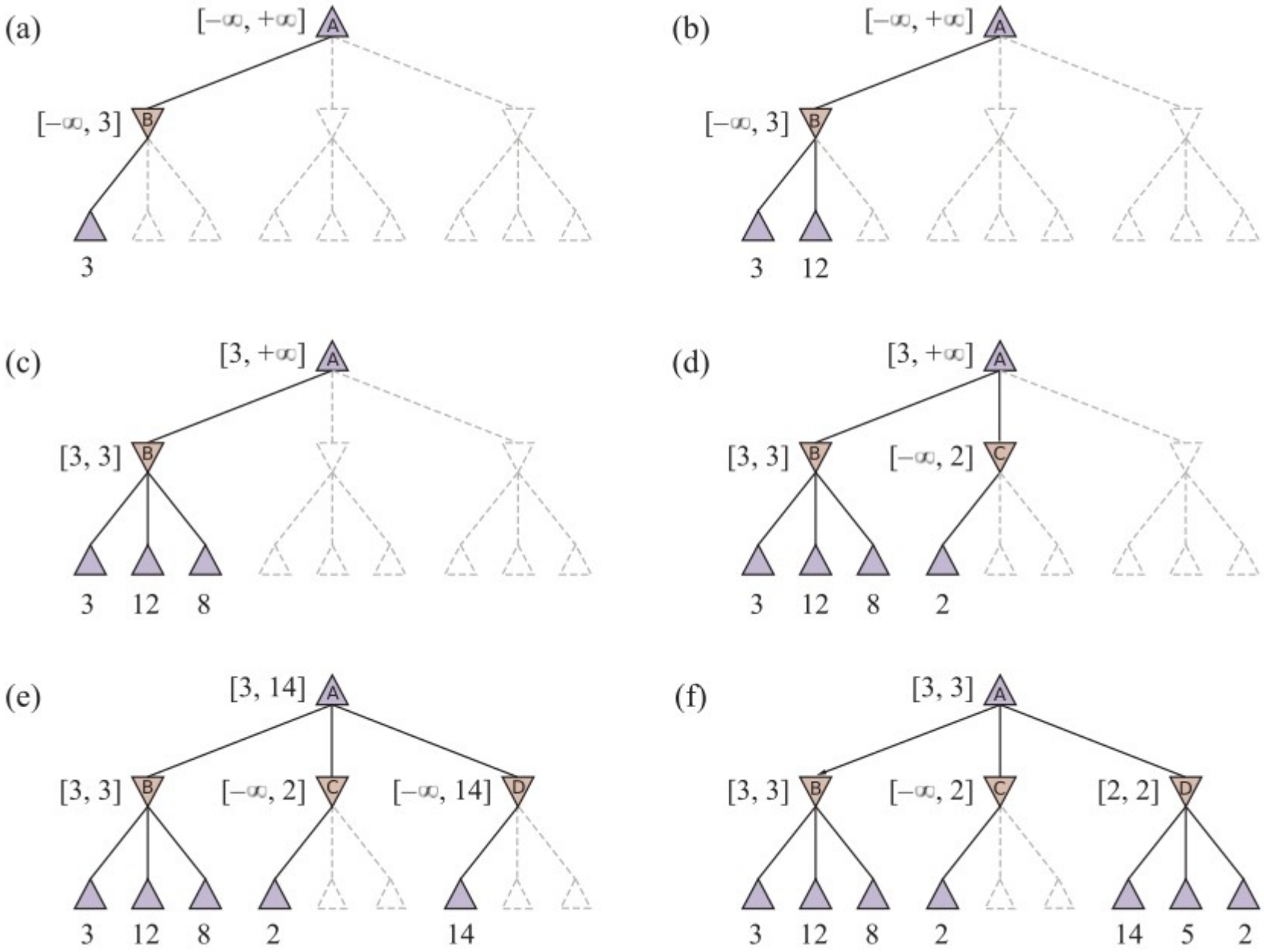
Should you encounter the min value, stop.
Disregard other subtrees

Alpha-delta search example

- Consider the generation of alpha/betas values for game:



Alpha-delta search example



Alpha-delta search complexity

- Effectiveness highly dependent on order of states
- If best successor it always at the end, technique is equiv. to minimax
- If states ordered perfectly, then time complexity becomes $O(b^{m/2})$
- Perfect move ordering is almost always impossible
  - Heuristics are still effective
  - e.g., in chess --- ordering function: start with captures, then threats, then forward moves, and then backward moves
  - $O(2 \cdot b^{m/2})$

Reading

- Sections 5.2.1 - 5.2.4

